

03

Infrastructure, Setup & Deployment

Every account, setting, secret and command needed to run TrustHome PH on free plans only. It covers onboarding a teammate, deploying a release, backing up without Blaze, and rolling back.

Revision	Rev 2.0 — 1 October 2026
Supersedes	Firebase Setup Plan
Stack baseline	React + Firebase Spark (Auth, Firestore, Hosting) + Cloudinary Free + Vercel Hobby API + GitHub Actions. No paid plan, no credit card.
Audience	Andresa (project owner), whoever deploys, every developer setting up a machine
Team	Andresa (Scrum Master) · Carillo · Laurente · Prades · Solpico

READ THIS FIRST

Four free services run TrustHome PH, and none needs a credit card. Firebase Spark provides Auth, Firestore and Hosting. Cloudinary Free stores photos and private documents. Vercel Hobby runs the trusted API. GitHub Actions runs the scheduled jobs. This document replaces the Firebase Setup Plan's "test mode" and "share the key in a DM" steps with safer practice. There are separate least-privilege service accounts per consumer, secrets live only in host settings, rules and indexes are deployed from Git, and a JSON backup runs before every migration. The step-by-step setup of the new API is in Section 5. Section 6 is the credential rotation that closes F-01.

CONTENTS

- | | | | |
|--|--------------------------------------|---|--|
| 1. Service inventory | 4. Cloudinary setup | 6. Secrets and service accounts (closes F-01) | 9. Release deployment runbook |
| 2. Environments | 5. Trusted API on Vercel (new, v1.4) | 7. Recommended CI (free) | 10. Quota monitoring |
| 3. Firebase setup (current, corrected) | | 8. Backups without Blaze | 11. Onboarding checklist (new machine) |

Document set: 01 Product Requirements & Scope · 02 System Architecture & Data Design · **03 Infrastructure, Setup & Deployment** · 04 Security & Privacy · 05 UI/UX Design System · 06 Roadmap & Release Plan · 07 QA & Testing Strategy

1. Service inventory

Service	Plan	Card?	Used for	Free limits that matter	Account owner
Firebase (<code>trusthome-ph</code>)	Spark	No	Auth, Firestore (<code>asia-southeast1</code>), Hosting, Rules	Firestore: 50K reads, 20K writes, 20K deletes per day; 1 GiB stored. Hosting: 10 GB stored, 360 MB/day transfer	Andresa (Owner); others Editor
Cloudinary	Free	No	Listing photos (public), verification documents (authenticated)	Monthly credits shared by storage, transformations and bandwidth	Andresa (Admin); Carillo (Technical)
Vercel	Hobby	No	Trusted API (<code>api-server/</code>)	1M function invocations, 4 h active CPU, 100 GB transfer per month; non-commercial use	Andresa; deploys from GitHub
GitHub	Free	No	Code, PRs, Actions (scraper, auto-complete, trust recompute)	Public repo: free standard runners. Private: 2,000 min/month	Repo admins: Andresa, Solpico
OpenStreetMap / Nominatim	Free	No	Map tiles, geocoding	Max 1 request/second, identifying User-Agent/Referer, attribution	—

BLOCKER · **Never** click "Upgrade to Blaze", "Enable billing" or "Start free trial" on any Google Cloud screen. Several console wizards (Cloud Functions, Storage, Cloud Build) push you there. If a feature asks for billing, it is out of scope. Find the free path in this document or ask the team.

2. Environments

Environment	What	Who uses it	Data
Local	Vite dev server + Firebase Emulator Suite (Auth, Firestore) + <code>vercel dev</code> for the API	Every developer	Seeded fake data; reset freely
Test/Live	<code>trusthome-ph</code> project, <code>trusthome-ph.web.app</code> , Vercel production URL	QA runs, demo, defense	Synthetic seed data only

BEST PRACTICE · Develop and test against **emulators** by default. That saves Spark quota and avoids damaging the demo data. Only QA runs, smoke tests and the defense touch the live project. A separate `trusthome-ph-dev` Firebase project (also free) is the standard next step after the defense.

3. Firebase setup (current, corrected)

3.1 Project and members

1. Console → *Project settings* → *Users and permissions*. Owner: one person. Everyone else: **Editor**. Remove anyone who has left the team.
2. Keep **Google Analytics disabled** (not needed, fewer data-processing disclosures).

3.2 Authentication

- Sign-in methods: **Email/Password** and **Google** enabled. Phone auth stays off (SMS is not free).
- *Settings* → *Authorized domains*: keep only `localhost`, `trusthome-ph.web.app`, `trusthome-ph.firebaseio.com`. Remove stale preview domains.
- *Settings* → *User actions*: leave "Email enumeration protection" **on**.
- Admins are created by setting `role: "admin"` on their `users/{uid}` document **in the console**, never from the app.

3.3 Firestore

- Location `asia-southeast1` (fixed forever, cannot change).
- **Never use "test mode" again**. Rules and indexes live in Git and are deployed only with the CLI:

```
firebase deploy --only firestore:rules,firestore:indexes # from repo root, after emulator tests pass
```

- After every rules deploy, record the commit hash in the progress report (e.g., `a1fe48e` on 1 Oct).

3.4 Hosting

```
npm ci && npm run build # builds with .env.production
firebase deploy --only hosting # live at https://trusthome-ph.web.app
```

`.env.production` (committed as `.env.production.example` with blank values):

```
VITE_FIREBASE_API_KEY=... # public client identifier - not a secret, rules protect data
VITE_FIREBASE_AUTH_DOMAIN=trusthome-ph.firebaseio.com
VITE_FIREBASE_PROJECT_ID=trusthome-ph
VITE_FIREBASE_APP_ID=...
VITE_CLOUDINARY_CLOUD_NAME=...
VITE_CLOUDINARY_PHOTO_PRESET=trusthome_uploads
VITE_API_BASE_URL=https://<your-project>.vercel.app
```

BEST PRACTICE · Restrict the browser API key in Google Cloud Console → *APIs & Services* → *Credentials*. Add HTTP-referrer restrictions (`trusthome-ph.web.app/*`, `trusthome-ph.firebaseio.com/*`, `localhost:*`), and limit the APIs to Identity Toolkit, Token Service and Firestore. This is free, and it stops other sites from using your quota.

4. Cloudinary setup

4.1 Folders and tags (already done on 1 Oct)

Dynamic Folders are enabled. Use `trusthome/listings/{listingId}` for photos, `trusthome/private/{uid}` for documents, and `trusthome/legacy-unclassified` for old root assets. Tag every asset with its purpose (`photo`, `verification`, `ownership`, `govId`). **Folders and tags organise assets. They do not make anything private.**

4.2 Presets

Preset	Signing	Delivery type	Allowed formats	Other settings
<code>trusthome_uploads</code> (photos)	Unsigned	<code>upload</code> (public)	jpg, jpeg, png, webp	Unique filename on; incoming transformation <code>c_limit,w_2560,h_2560</code> ; tags <code>photo</code>
<code>trusthome_private_docs</code> (new)	Signed	<code>authenticated</code>	jpg, jpeg, png, pdf	Unique filename on; tags <code>verification</code> ; no transformations

How to add the signed preset: *Settings* → *Upload* → *Upload presets* → *Add*. Name it `trusthome_private_docs`, set Signing mode to **Signed**, then *Upload control* → *Delivery type*: *Authenticated*, and set allowed formats as in the table.

- Remove **PDF** from the unsigned photo preset. Otherwise someone can upload documents through the public path (TC-DOC-006).
- On free accounts, delivery of PDF and ZIP files can be blocked by default. If PDF previews fail, check *Settings* → *Security* → "Allow delivery of PDF and ZIP files". Private documents go through the download API, but photos and legacy PDFs may still depend on this setting.
- Delete the `trusthome/_smoke-test` 1×1 test image from 1 Oct.

4.3 Keys

Settings → *API Keys*: create a key named `trusthome-api`, used only by Vercel. The **API secret goes only into Vercel environment variables**. It must never appear in the frontend, Git, chat or screenshots.

5. Trusted API on Vercel (new, v1.4)

5.1 One-time setup (about 30 minutes, Andresa + one developer)

1. Sign up at vercel.com with "Continue with GitHub" and choose the **Hobby** plan. No card is requested.
2. *Add New* → *Project* → *Import* the `trusthome-ph` repository.
3. **Root Directory**: `api-server`. **Framework preset**: Other. Leave the build command empty.
4. Add the environment variables in 5.2 for **Production** and **Preview**.
5. Deploy. Open `https://<project>.vercel.app/api/health`. It should return `{"status":"ok",...}` (TC-API-001).
6. Put the URL in the frontend's `VITE_API_BASE_URL`, rebuild, and redeploy Hosting.
7. *Settings* → *Git* → *Production branch* = `main` (or `dev` until release). Pull requests get preview URLs automatically.

5.2 Environment variables

Name	Value	Notes
<code>FIREBASE_PROJECT_ID</code>	<code>trusthome-ph</code>	
<code>FIREBASE_CLIENT_EMAIL</code>	<code>trusthome-api@trusthome-ph.iam.gserviceaccount.com</code>	From the API's own service account (6.2)

Name	Value	Notes
FIREBASE_PRIVATE_KEY	-----BEGIN PRIVATE KEY-----\n...	Paste with <code>\n</code> escapes; the code converts them
CLOUDINARY_CLOUD_NAME	your cloud name	
CLOUDINARY_API_KEY / CLOUDINARY_API_SECRET	from the <code>trustrhome-api</code> key	Secret: mark as Sensitive
ALLOWED_ORIGINS	<code>https://trustrhome-ph.web.app,https://trustrhome-ph.firebaseio.com,http://localhost:5173</code>	Comma-separated, no trailing slash
DOC_LINK_TTL_SECONDS	300	Lifetime of document links

5.3 Project files

```
// api-server/package.json
{ "name": "trustrhome-api", "private": true, "type": "module",
  "engines": { "node": "22.x" },
  "dependencies": { "firebase-admin": "13.x.x", "cloudinary": "2.x.x" },
  "scripts": { "test": "node --test" } }
```

```
// api-server/vercel.json
{ "regions": ["sin1"],
  "functions": { "api/**/*.js": { "maxDuration": 10 } },
  "headers": [{ "source": "/api/(.*)", "headers": [
    { "key": "Cache-Control", "value": "no-store" },
    { "key": "X-Content-Type-Options", "value": "nosniff" } ] } ] }
```

Replace the `x.x` placeholders with the exact current versions when you install them, and commit the lockfile.

5.4 Local development

```
cd api-server && npm ci
firebase emulators:start --only auth,firestore          # other terminal, from repo root
FIRESTORE_EMULATOR_HOST=127.0.0.1:8080 FIREBASE_AUTH_EMULATOR_HOST=127.0.0.1:9099 npx vercel dev
```

With the emulator variables set, `firebase-admin` talks to the emulators and needs no real key. Port the legacy `functions/` emulator tests into `api-server/test/` so the concurrency proof keeps running on every change.

NOTE · Fallback host. If Vercel is ever unavailable to the team, the same handlers can run as a small Express app on another free Node host. Only `lib/http.js` changes. Keep handlers free of Vercel-specific APIs to preserve that option (and the Laravel path in 02 ADR-003).

6. Secrets and service accounts (closes F-01)

6.1 Rules

- **One service account per consumer:** `trusthome-api` (Vercel) and `trusthome-jobs` (GitHub Actions). If one leaks, you revoke only that one.
- **Least privilege:** both get only **Cloud Datastore User** (`roles/datastore.user`). Verifying ID tokens needs no extra role. Do not reuse the default `firebase-adminsdk` account, which is over-privileged.
- Keys exist only in: Vercel env vars, GitHub Actions secrets, and (temporarily) one maintainer's password manager. **Never** in Git, chat apps, Google Drive or email.
- `.gitignore` must contain `serviceAccountKey*.json`, `*.pem`, `.env*` (except `*.example`).

6.2 Rotation procedure (do this in v1.4, day 1)

1. Google Cloud Console → IAM & Admin → Service Accounts → **Create** `trusthome-jobs` and `trusthome-api` with role `Cloud Datastore User`.
2. For each: Keys → Add key → JSON. Download it, and store it in Vercel or GitHub Secrets **immediately**. Then delete the local file.
3. Update the GitHub secret `FIREBASE_SERVICE_ACCOUNT` with the new `trusthome-jobs` JSON. Re-run the workflow and confirm it succeeds.
4. Deploy the API with the `trusthome-api` values and check `/api/health` plus one authenticated call.
5. On the **old** `firebase-adminsdk` account: Keys → **delete every user-managed key** that was ever shared or committed.
6. Prove it: run a tiny script with the old key file. It must fail with `invalid_grant` (TC-OPS-001).
7. Scan history: `gitleaks detect --source . --log-opts="--all"` (or `git log --all -p -- '*serviceAccount*'`). Record the result (TC-OPS-002).

BEST PRACTICE • Once a key is revoked, a leak in Git history is **harmless**: the key no longer works. Rewriting history (BFG, `git filter-repo`) is optional cleanup, and it forces every teammate to re-clone. Revocation is the real fix, so do it first.

6.3 GitHub Actions: safer secret handling

```
# .github/workflows/scrape.yml (relevant parts)
on:
  schedule: [{ cron: "0 18 * * *" }]      # daily 02:00 Asia/Manila (was weekly)
  workflow_dispatch: {}
permissions: { contents: read }
concurrency: { group: scheduled-jobs, cancel-in-progress: false }
jobs:
  run:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11", cache: pip }
      - run: pip install -r bank_scraper/requirements.txt
      - name: Write service account key
        env: { SA_JSON: "${{ secrets.FIREBASE_SERVICE_ACCOUNT }}" }
        run: printf '%s' "$SA_JSON" > bank_scraper/serviceAccountKey.json
      - run: python bank_scraper/scrapper.py
      - run: python bank_scraper/complete_expired_bookings.py
      - run: python bank_scraper/recompute_trust_scores.py
      - name: Remove key
        if: always()
        run: rm -f bank_scraper/serviceAccountKey.json
```

Why: the old `echo '${{ secrets... }}'` pasted the secret into the shell script itself, which breaks on quotes and risks exposure. Passing it through `env` is the documented safe pattern. Daily cadence fixes the "booking stays Confirmed for days" gap (TC-JOB-003). Use your actual script names.

WATCH OUT · GitHub **disables scheduled workflows after 60 days without repository activity** in public repos. Before the defense and after any long break, open *Actions* and check that the workflow is still enabled.

7. Recommended CI (free)

Add `.github/workflows/ci.yml` running on every pull request to `dev` and `main`: `npm ci → npm run lint → node --test → npm run build, cd api-server && npm ci && npm test`, and `python -m unittest discover -s bank_scraper -p "test_*.py"`. Protect `main` so it only accepts PRs with green CI and one approval. This turns the "current code gates pass" claim into something checked on every change.

8. Backups without Blaze

Managed Firestore export/import requires billing. Use an Admin-SDK JSON dump instead. It costs one read per document, so it is fine at this data size.

```
# bank_scraper/backup_firestore.py (run from a trusted machine)
import json, datetime, firebase_admin
from firebase_admin import credentials, firestore
firebase_admin.initialize_app(credentials.Certificate("serviceAccountKey.json"))
db = firestore.client()
COLS = ["users", "listings", "listingPrivate", "bookings", "ratings", "disputes", "trustScores",
        "notifications", "bankProperties", "scrapeJobs"]
stamp = datetime.datetime.now().strftime("%Y%m%d-%H%M")
def enc(v):
    if hasattr(v, "isoformat"): return {"__ts__": v.isoformat()}
    return str(v)
with open(f"backup-{stamp}.jsonl", "w", encoding="utf-8") as f:
    for col in COLS:
        n = 0
        for doc in db.collection(col).stream():
            f.write(json.dumps({"c": col, "id": doc.id, "d": doc.to_dict()}, default=enc) + "\n"); n += 1
        print(col, n)
```

- Run it **before** every migration, rules change, or bulk seed (NFR-REL-02). Compare the printed counts with the console (TC-OPS-003).
- The file contains personal data. Store it in a private, access-controlled location, and delete old backups after the defense.
- Write a matching `restore_firestore.py` that converts `__ts__` back to Timestamps and runs in dry-run mode by default.

9. Release deployment runbook

Do these steps **in order**. Stop if any check fails.

#	Step	Command / action	Check
1	Freeze	Merge <code>dev</code> → <code>release/x.y</code> PR; CI green	No open Critical defects
2	Backup	<code>python bank_scraper/backup_firestore.py</code>	Counts match console
3	Rules + indexes	<code>firebase deploy --only firestore:rules,firestore:indexes</code>	Index build finished (console)
4	API	Merge to the Vercel production branch	<code>/api/health</code> 200; TC-API-002/003
5	Migrations	Dry run → review → <code>--apply</code>	Script report saved
6	Frontend	<code>npm run build && firebase deploy --only hosting</code>	Site loads; version in footer
7	Smoke	TC-E2E-001 with the test-account matrix (07 §5)	All critical paths pass
8	Tag	<code>git tag v1.4.0 && git push --tags</code> ; update progress report	Release notes written

Rollback: Hosting → console *Release history* → *Roll back*. API → Vercel *Deployments* → *Promote previous* (instant). Rules → redeploy the previous commit's `firestore.rules`. Data → restore from step 2's backup (last resort).

10. Quota monitoring

Where	What to watch	Action threshold
Firebase → Usage and billing → Firestore	Daily reads/writes	> 25K reads on a normal day: look for unbounded <code>onSnapshot</code> listeners or missing <code>limit()</code>
Firebase → Hosting	Daily transfer	> 200 MB/day: check image sizes (serve Cloudinary <code>f_auto,q_auto,w_...</code>)
Cloudinary → Dashboard	Credits used	> 60%: add <code>f_auto,q_auto</code> to every photo URL and stop uploading full-size originals
Vercel → Usage	Invocations, active CPU	Unexpected spikes = retry loops in the client

BEST PRACTICE · Quota hygiene: paginate every list (`limit(20)` + cursor), unsubscribe listeners in `useEffect` cleanups, avoid real-time listeners on large public collections, and cache the trust-score collection instead of recomputing in the browser.

11. Onboarding checklist (new machine)

1. Clone the repo; install Node 22 LTS, Python 3.11, Firebase CLI, Vercel CLI (optional).
2. Copy `.env.example` → `.env.local`. Get the public values from the team lead (no secrets needed for frontend work).
3. `npm ci`, `firebase emulators:start`, then seed the emulator: `FIRESTORE_EMULATOR_HOST=127.0.0.1:8080 python bank_scraper/seed_demo_data.py` (the Admin SDK writes to the emulator whenever that variable is set).
4. Read *01 Requirements*, *02 Architecture* §3–6, and *05 Design System*. Run `npm test` before your first PR.